МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

Лабораторная работа № 2

по дисциплине «Математические основы баз данных»

по теме: «Базовые SQL-запросы и фильтрация данных»

Выполнил студент группы:
БВТ2505
Пондяков Игорь Дмитриевич
Проверил:
Быватов Семён Юрьевич

Москва 2026

Цель:

Наполнить спроектированную базу данных интернет-провайдера согласованными тестовыми данными и освоить базовые возможности языка SQL для выборки, фильтрации и сортировки информации в СУБД PostgreSQL.

Задачи:

1. Генерация тестовых данных: Наполнить все таблицы базы (включая справочники и крупные таблицы договоров, платежей и заявок) согласованными данными, используя функцию generate_series и генераторы случайных значений для создания объема не менее 1000 строк в ключевых сущностях.
2. Выборка уникальных значений: Составить запрос с использованием ключевого слова DISTINCT и псевдонимов (AS) для получения списка уникальных характеристик оборудования или услуг без дубликатов.
3. Сложная фильтрация: Реализовать запрос с комбинацией условий AND, OR и скобок для поиска абонентов или договоров по нескольким критериям одновременно (например, статус договора + цена тарифа + название услуги).
4. Применение предикатов: Использовать в отдельных запросах каждый из следующих предикатов хотя бы один раз: BETWEEN — для выбора договоров/платежей в определенном временном диапазоне; IN — для фильтрации по списку конкретных тарифов или статусов; LIKE / ILIKE — для поиска абонентов или сотрудников по части имени/должности; IS NULL — для выявления заявок без назначенного исполнителя или договоров без даты окончания.
5. Множественная сортировка: Написать запрос с секцией ORDER BY по двум столбцам в разных направлениях (например, сначала по убыванию стоимости тарифа, затем по возрастанию ФИО абонента).
6. Постраничный вывод: Реализовать пагинацию результатов выборки товаров/абонентов с помощью операторов LIMIT и OFFSET.
7. Документирование: Оформить тексты всех выполненных запросов, их назначение и фрагменты полученных результатов в отчете.

Выполнение

1. Генерация данных:

Для наполнения базы использована функция generate_series, позволяющая генерировать последовательности чисел для массового создания строк. В крупные таблицы внесено не менее 1000 записей:

subscribers: 1000 абонентов с уникальными паспортными данными и контактами.

contracts: 1000 договоров со случайным распределением тарифов и статусов (80% active, 10% suspended, 10% closed).

payments: ~3000 платежей (по 3 платежа на каждый активный договор).

Equipment: 1500 единиц оборудования для активных договоров.

support_tickets: 500 заявок в техническую поддержку.

Справочники (tariffs, service_categories, employees, services) заполнены вручную для обеспечения семантической согласованности данных. Генерация данных гарантирует отсутствие нарушений ссылочной целостности: все внешние ключи ссылаются на существующие записи родительских таблиц.

2. Запрос с DISTINCT и псевдонимами

Составлен запрос для получения списка уникальных моделей оборудования. Использование DISTINCT устраняет дубликаты (например, если выдано 500 роутеров одной модели, она будет показана один раз). Псевдоним AS equipment_model улучшает читаемость результата.

```
SELECT DISTINCT model AS equipment_model
FROM equipment
ORDER BY equipment_model;
```

3. Фильтрация по нескольким условиям (AND, OR, скобки)

Реализован сложный поиск активных абонентов, у которых либо дорогой тариф (>800 руб.), либо подключена услуга «Статический IP», при этом имя должно начинаться на «Абонент 1». Скобки критически важны: они группируют условие (цена > 800 OR услуга), которое затем объединяется с status = 'active' через AND. Без скобок логика была бы нарушена.

```
SELECT sub.full_name, tar.name AS tariff_name, tar.monthly_price
```

```
FROM subscribers sub
JOIN contracts con ON sub.subscriber_id = con.subscriber_id
JOIN tariffs tar ON con.tariff_id = tar.tariff_id
LEFT JOIN contract_services cs ON con.contract_id = cs.contract_id
LEFT JOIN services serv ON cs.service_id = serv.service_id
WHERE con.status = 'active'
  AND (tar.monthly_price > 800 OR serv.name = 'Статический IP')
  AND sub.full_name LIKE 'Абонент 1%';
```

4. Применение предикатов BETWEEN, IN, LIKE/ILIKE, IS NULL

BETWEEN — выбор договоров первого квартала 2026 года:

```
SELECT contract_id, start_date, status
FROM contracts
WHERE start_date BETWEEN '2026-01-01' AND '2026-03-31';
```

IN — фильтрация тарифов по списку названий:

```
SELECT name, monthly_price
FROM tariffs
WHERE name IN ('Старт', 'Геймер');
```

ILIKE — регистронезависимый поиск сотрудников:

```
SELECT full_name, position
FROM employees
WHERE position ILIKE '%инженер%';
```

IS NULL — заявки без назначенного исполнителя:

```
SELECT ticket_id, description, created_at
FROM support_tickets
WHERE employee_id IS NULL;
```

5. Сортировка по двум столбцам в разных направлениях

Запрос выводит активные договоры, отсортированные сначала по убыванию стоимости тарифа (DESC), а при равных ценах — по алфавиту имени абонента (ASC). Это стандартный паттерн для упорядоченных списков.

```
SELECT sub.full_name, tar.monthly_price, con.start_date
FROM contracts con
JOIN subscribers sub ON con.subscriber_id = sub.subscriber_id
JOIN tariffs tar ON con.tariff_id = tar.tariff_id
WHERE con.status = 'active'
ORDER BY tar.monthly_price DESC, sub.full_name ASC;
```

6. Постраничный вывод (LIMIT и OFFSET)

Реализована пагинация: вывод второй страницы списка абонентов при размере страницы 20 записей. OFFSET 20 пропускает первую страницу, LIMIT 20 ограничивает выборку.

```
SELECT subscriber_id, full_name, phone
FROM subscribers
ORDER BY subscriber_id
LIMIT 20 OFFSET 20;
```

Единый листинг

```
INSERT INTO service_categories (name, parent_id) VALUES
('Интернет', NULL), ('Телевидение', NULL), ('Безопасность', NULL);

INSERT INTO service_categories (name, parent_id)
SELECT 'Оптика', category_id FROM service_categories WHERE name =
'Интернет';
INSERT INTO service_categories (name, parent_id)
SELECT 'HD Каналы', category_id FROM service_categories WHERE name =
'Телевидение';

INSERT INTO tariffs (name, speed_down, speed_up, monthly_price,
connection_type) VALUES
```

```
('Старт', 50, 20, 450.00, 'Оптика'),
('Домашний', 100, 50, 600.00, 'Оптика'),
('Геймер', 300, 300, 900.00, 'Оптика'),
('Бизнес', 500, 500, 1500.00, 'Ethernet');

INSERT INTO employees (full_name, position, phone) VALUES
('Иванов И.И.', 'Инженер', '+79001112233'),
('Петрова А.С.', 'Оператор', '+79004445566'),
('Сидоров В.В.', 'Менеджер', '+79007778899');

INSERT INTO services (category_id, name, monthly_price)
SELECT category_id, 'Антивирус', 100.00 FROM service_categories WHERE
name = 'Безопасность';
INSERT INTO services (category_id, name, monthly_price)
SELECT category_id, 'Статический IP', 150.00 FROM service_categories
WHERE name = 'Интернет';
INSERT INTO services (category_id, name, monthly_price)
SELECT category_id, 'ТВ Пакет Спорт', 200.00 FROM service_categories
WHERE name = 'HD Каналы';

INSERT INTO subscribers (full_name, passport, phone, email)
SELECT
   'Абонент ' || g,
   '1234 ' || LPAD(g::text, 6, '0'),
   '+7900' || LPAD((g % 10000)::text, 4, '0') || LPAD((g / 10000)::text, 2, '0'),
   'user' || g || '@example.com'
FROM generate_series(1, 1000) AS g;

INSERT INTO contracts (subscriber_id, tariff_id, start_date, status)
SELECT
   s.subscriber_id,
   (random() * 3 + 1)::int,
   CURRENT_DATE - (random() * 365)::int,
   CASE
      WHEN random() < 0.8 THEN 'active'
      WHEN random() < 0.9 THEN 'suspended'
      ELSE 'closed'
   END
FROM subscribers s;
```

```
INSERT INTO payments (contract_id, amount, payment_date,
payment_method)
SELECT
   c.contract_id, t.monthly_price,
   c.start_date + (n * interval '1 month'),
   CASE WHEN n % 2 = 0 THEN 'Банковская карта' ELSE 'Наличные' END
FROM contracts c
JOIN tariffs t ON c.tariff_id = t.tariff_id
CROSS JOIN LATERAL generate_series(1, 3) AS n
WHERE c.status != 'closed';

INSERT INTO equipment (contract_id, type, model, serial_number, issue_date)
SELECT
   c.contract_id,
   CASE WHEN random() < 0.7 THEN 'Роутер' ELSE 'ТВ-приставка' END,
   CASE WHEN random() < 0.7 THEN 'TP-Link Archer' ELSE 'MAG 250'
END,
   'SN-' || LPAD((row_number() OVER ())::text, 6, '0'),
   c.start_date
FROM contracts c WHERE c.status = 'active' LIMIT 1500;

INSERT INTO support_tickets (contract_id, employee_id, description,
created_at, status)
SELECT
   c.contract_id, (random() * 2 + 1)::int,
   CASE
      WHEN random() < 0.3 THEN 'Нет интернета'
      WHEN random() < 0.6 THEN 'Медленная скорость'
      ELSE 'Настройка ТВ'
   END,
   c.start_date + (random() * 100)::int * interval '1 day',
   CASE WHEN random() < 0.5 THEN 'new' WHEN random() < 0.8 THEN
'in_progress' ELSE 'done' END
FROM contracts c WHERE c.status = 'active' LIMIT 500;

INSERT INTO contract_services (contract_id, service_id, connected_at)
SELECT c.contract_id, s.service_id, c.start_date
FROM contracts c CROSS JOIN services s
```

```
WHERE random() < 0.2 AND c.status = 'active';

SELECT DISTINCT model AS equipment_model
FROM equipment ORDER BY equipment_model;

SELECT sub.full_name, tar.name AS tariff_name, tar.monthly_price
FROM subscribers sub
JOIN contracts con ON sub.subscriber_id = con.subscriber_id
JOIN tariffs tar ON con.tariff_id = tar.tariff_id
LEFT JOIN contract_services cs ON con.contract_id = cs.contract_id
LEFT JOIN services serv ON cs.service_id = serv.service_id
WHERE con.status = 'active'
  AND (tar.monthly_price > 800 OR serv.name = 'Статический IP')
  AND sub.full_name LIKE 'Абонент 1%';

SELECT contract_id, start_date, status
FROM contracts
WHERE start_date BETWEEN '2026-01-01' AND '2026-03-31';

SELECT name, monthly_price
FROM tariffs
WHERE name IN ('Старт', 'Геймер');

SELECT full_name, position
FROM employees
WHERE position ILIKE '%инженер%';

SELECT ticket_id, description, created_at
FROM support_tickets
WHERE employee_id IS NULL;

SELECT sub.full_name, tar.monthly_price, con.start_date
FROM contracts con
JOIN subscribers sub ON con.subscriber_id = sub.subscriber_id
JOIN tariffs tar ON con.tariff_id = tar.tariff_id
WHERE con.status = 'active'
ORDER BY tar.monthly_price DESC, sub.full_name ASC;

SELECT subscriber_id, full_name, phone
```

```
FROM subscribers
ORDER BY subscriber_id
LIMIT 20 OFFSET 20;
```

Вывод:

В ходе выполнения лабораторной работы №2 была наполнена спроектированная база данных интернет-провайдера тестовыми данными и реализован набор базовых SQL-запросов на выборку и фильтрацию в СУБД PostgreSQL.

В результате работы:

1. Все таблицы базы данных наполнены согласованными тестовыми данными с использованием функции generate_series. В крупные таблицы (subscribers, contracts, payments) внесено не менее 1000 строк, что обеспечивает репрезентативность результатов при выполнении аналитических запросов.
2. Составлен запрос с ключевым словом DISTINCT и псевдонимами столбцов, демонстрирующий устранение дубликатов при выборке уникальных значений.
3. Реализован сложный запрос с фильтрацией по нескольким условиям, объединённым операторами AND и OR со скобками, что подтверждает понимание приоритета логических операторов в SQL.
4. В отдельных запросах применены все требуемые предикаты: BETWEEN (диапазон дат), IN (список значений), ILIKE (регистронезависимый поиск по шаблону) и IS NULL (проверка отсутствия значения).
5. Написан запрос с сортировкой по двум столбцам в разных направлениях (DESC и ASC), демонстрирующий многоуровневое упорядочивание результатов.
6. Реализован постраничный вывод данных с помощью операторов LIMIT и OFFSET, имитирующий механизм пагинации клиентского приложения.

Фрагменты выполнения:

Query Query History

```
1  SELECT count(*) FROM subscribers; -- должно быть 1000
2  SELECT count(*) FROM contracts;   -- должно быть 1000
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page N

	count bigint
1	1000

Query Query History

```
1  SELECT DISTINCT model AS equipment_model
2  FROM equipment
3  ORDER BY equipment_model;
4
```

Data Output Messages Notifications

	equipment_model character varying (100)
1	MAG 250
2	TP-Link Archer

Query Query History

```
1  SELECT sub.full_name, tar.name AS tariff_name, tar.monthly_price
2  FROM subscribers sub
3  JOIN contracts con ON sub.subscriber_id = con.subscriber_id
4  JOIN tariffs tar ON con.tariff_id = tar.tariff_id
5  LEFT JOIN contract_services cs ON con.contract_id = cs.contract_id
6  LEFT JOIN services serv ON cs.service_id = serv.service_id
7  WHERE con.status = 'active'
8    AND (tar.monthly_price > 800 OR serv.name = 'Статический IP')
9    AND sub.full_name LIKE 'Абонент 1%';
10
```

Data Output Messages Notifications

Showing

	full_name character varying (150)	tariff_name character varying (100)	monthly_price numeric (10,2)
1	Абонент 12	Геймер	900.00
2	Абонент 114	Геймер	900.00
3	Абонент 115	Бизнес	1500.00
4	Абонент 170	Геймер	900.00
5	Абонент 172	Геймер	900.00
6	Абонент 15	Домашний	600.00
7	Абонент 100	Геймер	900.00
8	Абонент 116	Геймер	900.00
9	Абонент 117	Домашний	600.00
10	Абонент 125	Геймер	900.00

Query | Query History

```sql
SELECT name, monthly_price
FROM tariffs
WHERE name IN ('Старт', 'Геймер');
```

Data Output | Messages | Notifications

	name character varying (100)	monthly_price numeric (10,2)
1	Старт	450.00
2	Геймер	900.00

Query | Query History

```sql
SELECT contract_id, start_date, status
FROM contracts
WHERE start_date BETWEEN '2026-01-01' AND '2026-03-31';
```

Data Output | Messages | Notifications

	contract_id [PK] integer	start_date date	status character varying (20)
1	3	2026-01-20	active
2	6	2026-03-17	active
3	7	2026-02-21	active
4	12	2026-03-15	active
5	13	2026-01-24	active
6	16	2026-01-17	active
7	19	2026-02-01	active
8	25	2026-01-15	active
9	27	2026-02-05	active
10	28	2026-01-15	active

Query | Query History

```sql
SELECT full_name, position
FROM employees
WHERE position ILIKE '%инженер%';
```

Data Output | Messages | Notifications

	full_name character varying (150)	position character varying (100)
1	Иванов И.И.	Инженер

Query | Query History

```sql
SELECT ticket_id, description, created_at
FROM support_tickets
WHERE employee_id IS NULL;
```

Data Output | Messages | Notifications

ticket_id [PK] integer	description text	created_at timestamp with time zone

Query | Query History

```sql
SELECT sub.full_name, tar.monthly_price, con.start_date
FROM contracts con
JOIN subscribers sub ON con.subscriber_id = sub.subscriber_id
JOIN tariffs tar ON con.tariff_id = tar.tariff_id
WHERE con.status = 'active'
ORDER BY tar.monthly_price DESC, sub.full_name ASC;
```

Data Output | Messages | Notifications

	full_name character varying (150)	monthly_price numeric (10,2)	start_date date
1	Абонент 104	1500.00	2026-02-19
2	Абонент 108	1500.00	2026-02-08
3	Абонент 11	1500.00	2025-10-06
4	Абонент 115	1500.00	2025-10-02
5	Абонент 130	1500.00	2026-01-29
6	Абонент 137	1500.00	2025-10-19
7	Абонент 139	1500.00	2025-10-31
8	Абонент 14	1500.00	2026-09-14
9	Абонент 142	1500.00	2026-06-03
10	Абонент 143	1500.00	2026-05-11

Query | Query History

```sql
SELECT subscriber_id, full_name, phone
FROM subscribers
ORDER BY subscriber_id
LIMIT 20 OFFSET 20;
```

Data Output | Messages | Notifications

	subscriber_id [PK] integer	full_name character varying (150)	phone character varying (20)
1	21	Абонент 21	+7900002100
2	22	Абонент 22	+7900002200
3	23	Абонент 23	+7900002300
4	24	Абонент 24	+7900002400
5	25	Абонент 25	+7900002500
6	26	Абонент 26	+7900002600
7	27	Абонент 27	+7900002700
8	28	Абонент 28	+7900002800
9	29	Абонент 29	+7900002900
10	30	Абонент 30	+7900003000